

CEG4166 Lab 1

FreeRTOS Task Implementation Report

Board: NUCLEO-L552ZE-Q (STM32L552ZET6Q)

RTOS: FreeRTOS V10.3.1

Toolchain: GNU Tools for STM32 (GCC 13.3)

1. Introduction

This lab demonstrates FreeRTOS task implementation on an STM32L5 microcontroller. The project implements three concurrent tasks that showcase real-time scheduling, inter-task communication, and hardware interaction through GPIO and UART peripherals.

2. Part B - FreeRTOS Task Implementation

2.1 Task 1: SystemStatusTask

Purpose: Indicates system operational status through LED blinking.

Implementation Details:

- **GPIO Pin:** PC7 (Green LED)
- **Period:** 500 ms (1 Hz blink rate)
- **Delay Function:** `vTaskDelay(pdMS_TO_TICKS(500))`
- **Stack Size:** 128 bytes
- **Priority:** `tskIDLE_PRIORITY + 1` (Low)

Code:

```
void SystemStatusTask(void *argument)
{
    for (;;)
    {
```

```
{  
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_7);  
    vTaskDelay(pdMS_TO_TICKS(500));  
}  
}
```

Behavior: The task runs in an infinite loop, toggling the green LED state every 500ms. This creates a visual heartbeat indicating the system is running normally. When suspended (via button press), the LED stops blinking.

2.2 Task 2: LoggerTask

Purpose: Sends periodic status messages and event notifications via UART.

Implementation Details:

- **UART Interface:** LPUART1 (PG7=TX, PG8=RX)
- **Baud Rate:** 115200 bps, 8N1
- **Period:** 1000 ms (1 second)
- **Output Message:** [LOG] System running...
- **Stack Size:** 256 bytes
- **Priority:** `tskIDLE_PRIORITY + 2` (Higher than SystemStatusTask)

Code:

```
void LoggerTask(void *argument)  
{  
    char eventMsg[LOG_MSG_LEN];  
    const char periodicMsg[] = "[LOG] System running...\r\n";  
  
    for (;;)   
    {  
        /* Drain any queued event messages first */  
        while (xQueueReceive(logQueue, eventMsg, 0) == pdTRUE)  
        {  
            while (uartBusy) { vTaskDelay(1); }  
            uartBusy = 1;  
            HAL_UART_Transmit_IT(&hlpuart1,  
                                (uint8_t *)eventMsg,  
                                strlen(eventMsg));  
        }  
    }
```

```

    /* Send periodic log message */
    while (uartBusy) { vTaskDelay(1); }
    uartBusy = 1;
    HAL_UART_Transmit_IT(&hlpuart1,
                        (uint8_t *)periodicMsg,
                        strlen(periodicMsg));

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

Behavior: The task first checks a message queue for any pending event messages (from button presses), transmits them, then sends the periodic log message. Interrupt-driven UART transmission is used with a busy flag to prevent concurrent access.

3. Part C - User Button Handling

3.1 Task 3: ButtonHandlerTask

Purpose: Handles user button presses to suspend/resume LED blinking.

Implementation Details:

- **GPIO Pin:** PC13 (User Button, active low)
- **Interrupt:** EXTI13 (falling edge trigger)
- **Synchronization:** Binary semaphore (buttonSemaphore)
- **Event Message:** [EVENT] Button pressed
- **Stack Size:** 128 bytes
- **Priority:** `tskIDLE_PRIORITY + 3` (Highest)

Code:

```

void ButtonHandlerTask(void *argument)
{
    uint8_t suspended = 0;
    char msg[LOG_MSG_LEN] = "[EVENT] Button pressed\r\n";

    for (;;)
    {

```

```
xSemaphoreTake(buttonSemaphore, portMAX_DELAY);

if (suspended)
{
    vTaskResume(SystemStatusTaskHandle);
    suspended = 0;
}
else
{
    vTaskSuspend(SystemStatusTaskHandle);
    suspended = 1;
}

xQueueSend(logQueue, msg, 0);
}
```

Interrupt Handler (EXTI Callback):

```
void HAL_GPIO_EXTI_Falling_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13)
    {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        xSemaphoreGiveFromISR(buttonSemaphore, &xHigherPriorityTaskWoken);
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
```

Behavior:

- **First button press:** Suspends SystemStatusTask (LED stops), sends "[EVENT] Button pressed"
- **Second button press:** Resumes SystemStatusTask (LED resumes), sends "[EVENT] Button pressed"

4. Part D - Task Priorities and Observation

4.1 Task Priority Table

Task Name	Priority Level	Priority Value	Stack Size	Role
-----------	----------------	----------------	------------	------

ButtonHandlerTask	Highest	tskIDLE_PRIORITY + 3	128 bytes	User input handling
LoggerTask	High	tskIDLE_PRIORITY + 2	256 bytes	UART communication
SystemStatusTask	Low	tskIDLE_PRIORITY + 1	128 bytes	LED status indication
Idle Task	Lowest	tskIDLE_PRIORITY (0)	128 bytes	System idle

4.2 UART Output Screenshots

Expected Terminal Output (Normal Operation):

```
[LOG] System running...
[LOG] System running...
[LOG] System running...
[LOG] System running...
[LOG] System running...
```

Expected Terminal Output (With Button Presses):

```
[LOG] System running...
[LOG] System running...
[EVENT] Button pressed
[LOG] System running...
[LOG] System running...
[EVENT] Button pressed
[LOG] System running...
```

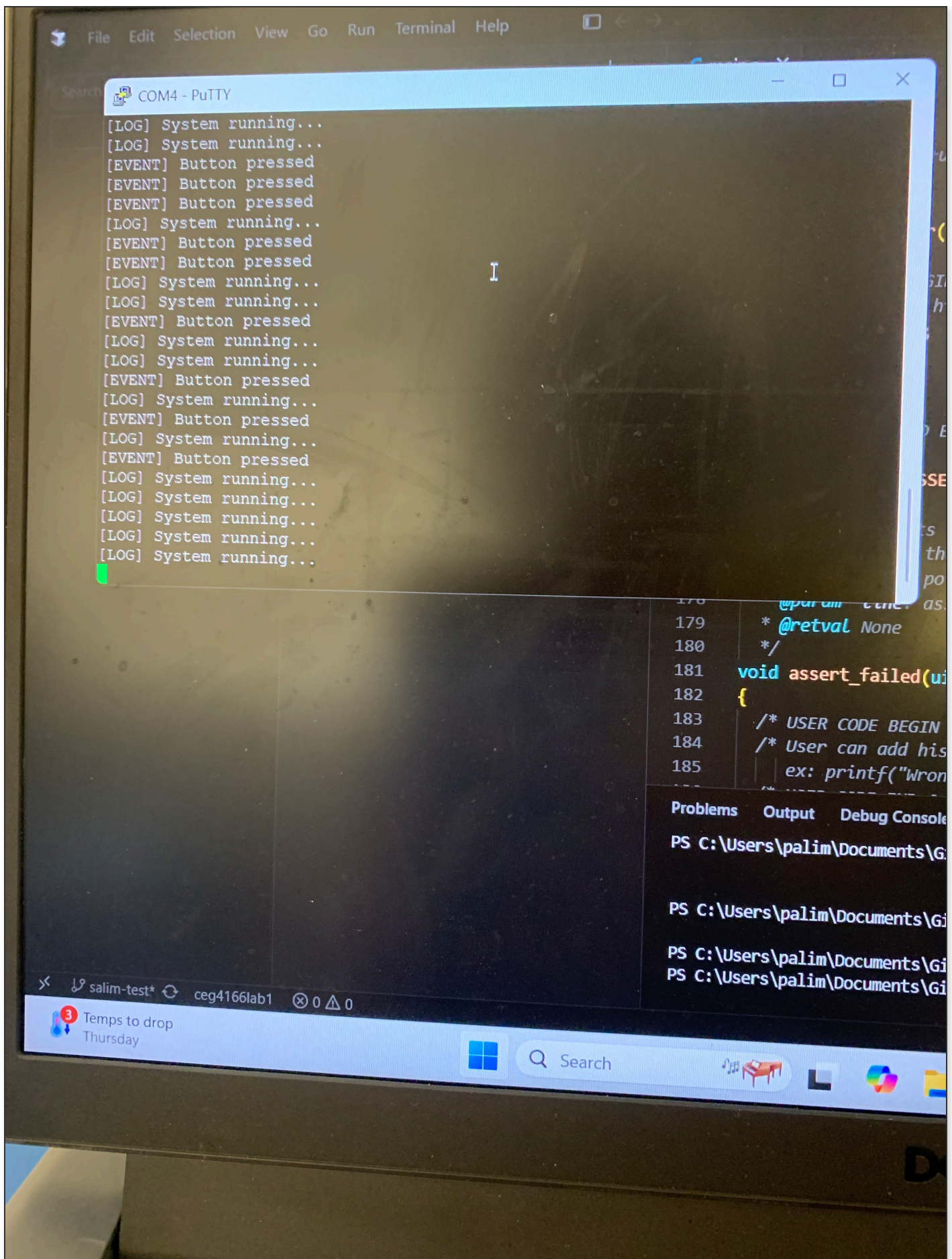


Figure 1: UART output showing [LOG] and [EVENT] messages in PuTTY terminal

4.3 FreeRTOS Scheduling Behavior Analysis

4.3.1 Preemptive Scheduling

FreeRTOS is configured with preemptive scheduling (`configUSE_PREEMPTION = 1`). This means:

- Higher priority tasks can interrupt lower priority tasks at any time
- When a higher priority task becomes ready (e.g., ButtonHandlerTask receives semaphore), it immediately preempts any running lower priority task
- The scheduler always runs the highest priority ready task

4.3.2 Observed Behavior with Priority Configuration

LoggerTask (Priority 2) vs SystemStatusTask (Priority 1):

- LoggerTask has higher priority than SystemStatusTask as required by Part D
- When LoggerTask is ready to transmit UART data, it preempts SystemStatusTask
- This ensures UART messages are sent promptly without being delayed by LED toggling
- However, both tasks spend most of their time blocked (in `vTaskDelay()`), so in practice they rarely compete for CPU time

LED Behavior Observation:

- The LED continues to blink at a consistent 1 Hz rate (500ms on/off)
- The LED timing is not noticeably affected by LoggerTask because:
 1. UART transmission is interrupt-driven (non-blocking)
 2. Both tasks use `vTaskDelay()` which yields to the scheduler
 3. Actual CPU time for each task is minimal (~microseconds per iteration)

UART Message Timing:

- Messages appear every 1 second as expected
- Button event messages appear immediately after button press (within ~1ms)
- The higher priority of LoggerTask ensures UART operations complete without being preempted by LED operations

4.3.3 Task State Transitions

Task	Running	Ready	Blocked	Suspended
SystemStatusTask	Toggling LED	After delay expires	During vTaskDelay()	After button press
LoggerTask	Sending UART	After delay expires	During vTaskDelay()	N/A
ButtonHandlerTask	Processing button	After semaphore given	Waiting for semaphore	N/A

4.3.4 Why Higher Priority for LoggerTask Matters

Assigning LoggerTask a higher priority than SystemStatusTask provides the following benefits:

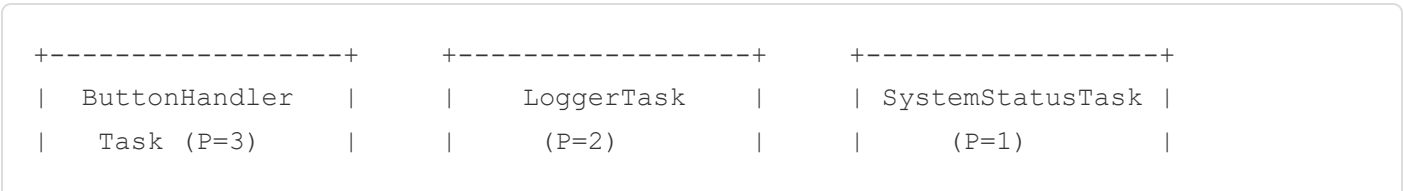
1. **Guaranteed UART Timing:** Log messages are transmitted without delay from LED operations
2. **Event Responsiveness:** Button press events are logged immediately since ButtonHandlerTask sends to the queue and LoggerTask (being higher priority than SystemStatusTask) processes it promptly
3. **Communication Priority:** In embedded systems, logging/communication is often more critical than visual indicators

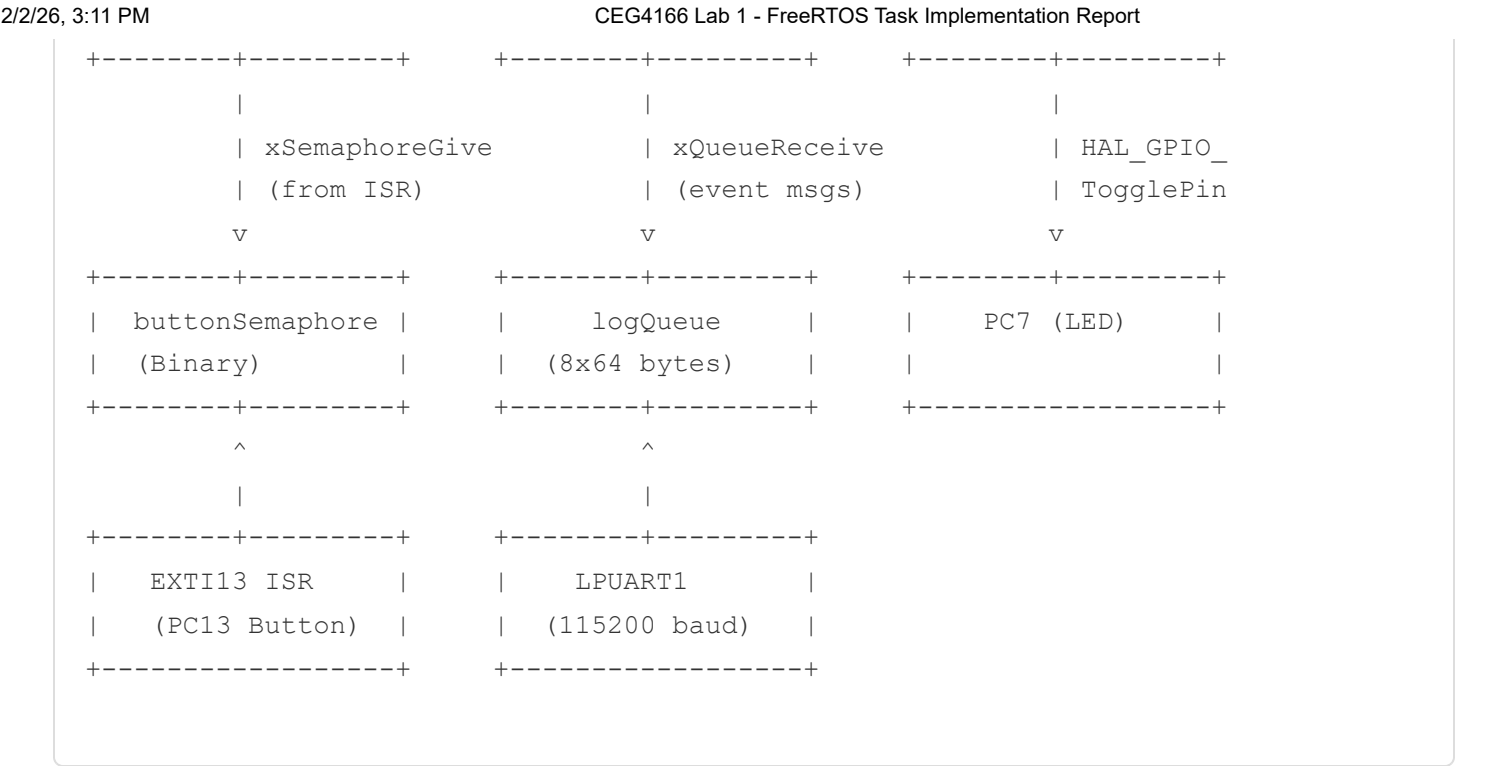
4.3.5 Scheduler Tick Configuration

```
configTICK_RATE_HZ = 1000    // 1ms tick period
configCPU_CLOCK_HZ = 110MHz  // System clock
configMAX_PRIORITIES = 56    // Maximum priority levels
```

With a 1ms tick rate, the scheduler can respond to task state changes within 1ms, providing responsive real-time behavior.

5. System Architecture Diagram





6. Conclusion

This lab successfully demonstrated:

- **FreeRTOS Task Creation:** Three concurrent tasks with different priorities
- **Task Synchronization:** Binary semaphore for ISR-to-task communication
- **Inter-Task Communication:** Message queue for event logging
- **Task Control:** Dynamic suspend/resume of tasks via user input
- **Priority-Based Scheduling:** Higher priority tasks preempt lower priority tasks
- **Hardware Integration:** GPIO for LED and button, UART for serial communication

The implementation correctly follows FreeRTOS best practices including ISR-safe API usage (`xSemaphoreGiveFromISR`), proper task priorities for responsiveness, and non-blocking UART transmission.

7. References

- FreeRTOS V10.3.1 Reference Manual
- STM32L5 HAL Driver Documentation
- NUCLEO-L552ZE-Q User Manual (UM2581)

- STM32CubeIDE User Guide